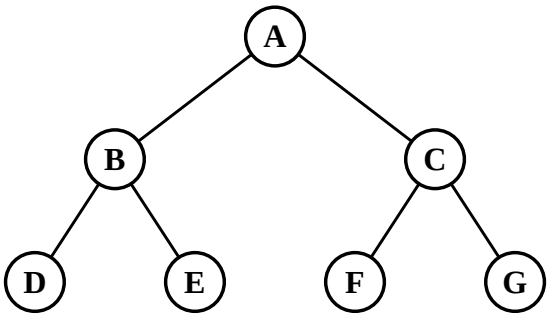
1. Question 1 – Breadth-First Search (BFS)

Graph: A is connected to B and C; B is connected to D and E; C is connected to F and G.



Approach: BFS uses a **Queue (FIFO)**. The front node is removed and visited, then its children are added at the rear from left to right. The Queue column shows the queue *after* each iteration.

Iteration	Visited Node (removed from front)	Queue (after iteration)
1	A	B, C
2	B	C, D, E
3	C	D, E, F, G
4	D	E, F, G
5	E	F, G
6	F	G
7	G	Empty

Answers

1. **BFS traversal order:** A → B → C → D → E → F → G
2. **Queue contents after each iteration:** [B, C] → [C, D, E] → [D, E, F, G] → [E, F, G] → [F, G] → [G] → [ ]

---

2. Depth-First Search (DFS) – same graph

Approach: DFS uses a **Stack (LIFO)**. The top node is popped and visited, then its children are pushed **right to left** so that the left child is on top and is explored first. The stack is written from **bottom to top** (top is the right-most item).

Iteration	Visited Node (popped from top)	Stack (after iteration, bottom → top)
1	A	C, B
2	B	C, E, D
3	D	C, E
4	E	C
5	C	G, F
6	F	G
7	G	Empty

DFS traversal order: A → B → D → E → C → F → G

---

3. Question 3 – BFS on the 8-Puzzle

Initial State	Goal State
1 3 6 5 0 2 4 7 8	1 2 3 4 5 6 7 8 0

Conventions used: the blank (0) is moved in the order **Up, Down, Left, Right**; a state that has already been generated is not added to the queue again. States are named S0, S1, S2, … in the order they are generated (S0 is the initial state). The goal is checked when a state is removed from the queue.

Iteration table (first 5 iterations)

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1	**S0**	S0	S1, S2, S3, S4
2	**S1**	S1, S2, S3, S4	S2, S3, S4, S5, S6
3	**S2**	S2, S3, S4, S5, S6	S3, S4, S5, S6, S7, S8
4	**S3**	S3, S4, S5, S6, S7, S8	S4, S5, S6, S7, S8, S9, S10
5	**S4**	S4, S5, S6, S7, S8, S9, S10	S5, S6, S7, S8, S9, S10, S11, S12

States generated in these iterations (blank moved Up / Down / Left / Right):

S0 (initial)	**S1** (S0 → Up)	**S2** (S0 → Down)	**S3** (S0 → Left)
1 3 6 5 0 2 4 7 8	1 0 6 5 3 2 4 7 8	1 3 6 5 7 2 4 0 8	1 3 6 0 5 2 4 7 8

S4 (S0 → Right) 1 3 6 5 2 0 4 7 8	**S5** (S1 → Left) 0 1 6 5 3 2 4 7 8	**S6** (S1 → Right) 1 6 0 5 3 2 4 7 8	**S7** (S2 → Left) 1 3 6 5 7 2 0 4 8
S8 (S2 → Right) 1 3 6 5 7 2 4 8 0	**S9** (S3 → Up) 0 3 6 1 5 2 4 7 8	**S10** (S3 → Down) 1 3 6 4 5 2 0 7 8	**S11** (S4 → Up) 1 3 0 5 2 6 4 7 8
S12 (S4 → Down) 1 3 6 5 2 8 4 7 0			

Iterations 2 to 5 skip the "move back" child because it is S0, which is already generated.

Answers

1. **Node expanded in each iteration:** Iteration 1 → S0, Iteration 2 → S1, Iteration 3 → S2, Iteration 4 → S3, Iteration 5 → S4. BFS keeps removing the front of the queue in the same FIFO order. The goal state is removed from the queue in iteration **311**, at depth 8.
2. **Total states generated: 500** states (the initial state plus 499 newly generated states) were generated, and 310 of them were expanded before the goal was reached.
3. **Complete solution path** (8 moves of the blank): Right → Up → Left → Down → Left → Down → Right → Right.

Step	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
Move	–	Right	Up	Left	Down	Left	Down	Right	Right
State	1 3 6 5 0 2 4 7 8	1 3 6 5 2 0 4 7 8	1 3 0 5 2 6 4 7 8	1 0 3 5 2 6 4 7 8	1 2 3 5 0 6 4 7 8	1 2 3 0 5 6 4 7 8	1 2 3 4 5 6 0 7 8	1 2 3 4 5 6 7 0 8	**1 2 3** **4 5 6** **7 8 0**

4. **Why BFS always finds the shortest solution in an unweighted 8-puzzle:** BFS expands all states at depth *d* before any state at depth *d*+1. Every move in the 8-puzzle costs the same (1), so the depth of a state is exactly its path cost. The first time the goal is reached, it is therefore at the smallest possible depth, and no shorter solution can exist. This also makes BFS complete (it always finds a solution if one exists) and optimal for uniform step cost. Here the shortest solution has 8 moves. The drawback is memory, since BFS must store a whole level of states in the queue.